RVT-Swarm: Recoverability-Verified Topological Control for Decentralized Swarm Formation

Anonymous Authors

***Abstract*—Decentralized swarm control is attractive because it scales to large teams, avoids a single point of failure, reduces communication and computation bottlenecks, and can operate from local observations rather than full global state. However, the same locality makes decentralized formation control difficult in cluttered environments: robots may avoid instantaneous collisions yet still lose formation structure, enter deadlock, or fail to recover after passing through bottlenecks. We propose RVT-Swarm, a decentralized graph-based controller that combines per-robot action prediction with discrete topology adaptation and a learned *recoverability* score map over candidate formation modes. The recoverability signal estimates whether a local swarm state can remain safe, continue making progress, and return toward a valid formation tube under each candidate topology. Methodologically, RVT-Swarm is new in both training and inference. During training, counterfactual rollout scores are distilled into structured supervision for a multi-head graph controller and optimized with an equal-weight active-objective loss. During inference, the learned score map drives recoverability-aware topology selection, latent formation geometry updates, and a lightweight safety projection for high-risk situations. In the best single-seed benchmark run currently used in this paper (400 evaluation episodes per method), RVT-Swarm achieves the best overall success among all compared methods at 45.5%, together with 58.5% goal reaching, 57.5% collision-free execution, and 52.0% formation satisfaction. It also yields the lowest deadlock rate (2.25%) and the lowest irreversible-collapse rate (39.25%) among the learned decentralized controllers. These results indicate that recoverability-aware topology control can improve decentralized swarm formation performance beyond existing learned and classical baselines in cluttered navigation settings.**

***Index Terms*—Multi-robot systems, swarm formation control, graph neural networks, safety-critical control, topology adaptation, recoverability.**

I. Introduction

Multi-robot formations are useful only when they remain both *safe* and *task-capable*. In warehouse logistics, cooperative inspection, or search-and-rescue, a team must not merely avoid collisions; it must also preserve enough structure to pass through clutter, regroup, and continue making progress toward a collective goal [1], [2], [3]. This is especially difficult in decentralized settings with limited local sensing, heterogeneous local contexts, and team sizes that vary at deployment time.

Existing approaches usually solve only part of this problem. Graph-based decentralized controllers scale well to large teams but often optimize local motion directly, without an explicit notion of whether the team can still *recover* a useful formation after local reconfiguration [4], [5]. Control-barrier-function (CBF) approaches provide strong instantaneous safety structure [6], [7], [8], but they typically reason about collision avoidance rather than return to a formation tube. Adaptive-formation approaches can change shape to fit the environment, but usually rely on hand-designed switching logic or implicit policies without an explicit recoverability score over alternative topologies [2], [3]. Reactive methods such as ORCA can maintain goal progress, but often do so by sacrificing formation quality entirely [9].

This paper explores a different angle: rather than asking only whether the current action is safe, we ask whether the current local state is *recoverable* under a small set of candidate formation topologies. In RVT-Swarm, each local graph observation is mapped not only to a control action but also to a score map over topological actions (KEEP, COMPRESS, LINE, SPLIT, RECOVER). The controller then evaluates these topologies counterfactually and selects among them using a fixed rank-based rule that compares recoverability, context, prior agreement, and switching readiness. An optional lightweight projection-based shield intervenes only when risk is high or when all candidate topologies appear unrecoverable.

The resulting system is deliberately pragmatic. It does not claim an exact viability kernel or a formal distributed reachability proof. Instead, it uses short-horizon counterfactual rollouts to construct a learned surrogate of recoverability and then deploys that surrogate online for topology adaptation. This design is consistent with the current codebase and the current benchmark setting.

Methodologically, the novelty is split across two coupled stages. During training, counterfactual rollout-score vectors are distilled into structured supervision for action, topology, score-map, and auxiliary heads, so the network learns not only what action to imitate but also how alternative topologies rank in recoverability. The resulting multi-head model is optimized with an equal-weight active-objective loss rather than a hand-balanced coefficient vector. During inference, the learned score map is used by a lexicographic topology selector, a latent formation geometry update, and a risk-triggered projection shield to decide both how the swarm should move and what formation structure it should adopt next.

We make the following contributions:

1) We formulate *recoverability-aware topology adaptation* for decentralized swarm formation, where candidate formation modes are scored by their short-horizon ability

Manuscript prepared from the released research prototype codebase.

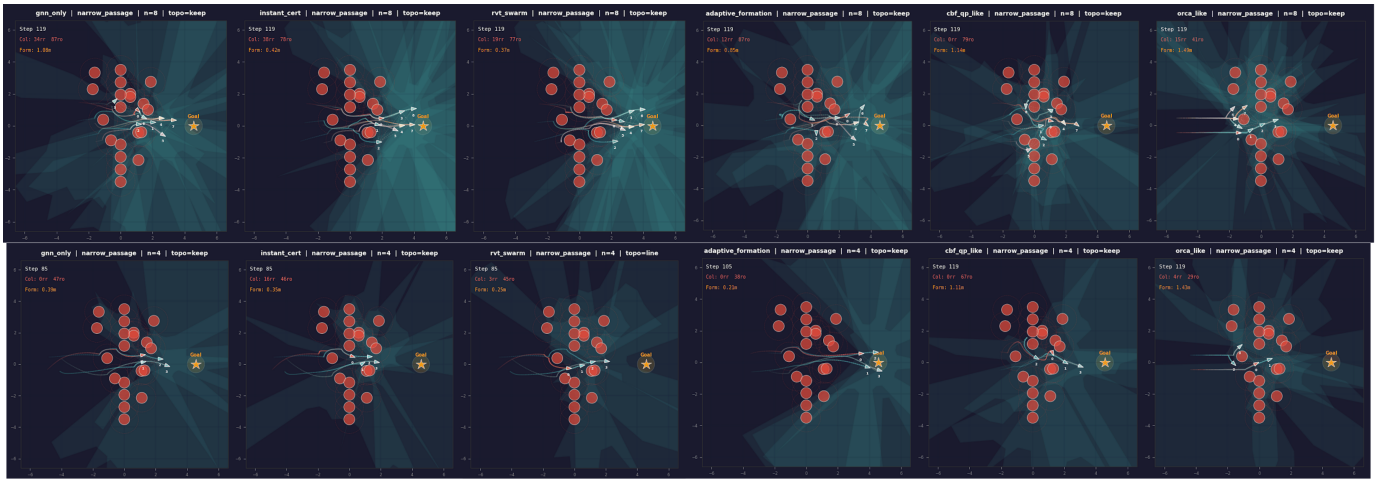

Fig. 1: Qualitative teaser for decentralized swarm formation in a cluttered narrow passage. Each column shows one method (`gnn_only`, `instant_cert`, `rvt_swarm`, `adaptive_formation`, `cbf_qp_like`, and `orca_like`) on the same representative scenario; the top row uses $n = 8$ robots and the bottom row uses $n = 4$. The overlays report per-rollout diagnostics already produced by the benchmark recorder, including step count, collision count, formation error, and active topology mode.

to remain safe, make goal progress, and return toward a formation tube.

2) We introduce a rollout-supervised training recipe for a multi-head graph-attention controller, where counterfactual topology rollouts are distilled into action, topology, score-map, and auxiliary targets, and the model is trained with an equal-weight active-objective loss plus gradient-scaled auxiliary coupling instead of hand-balanced loss weights.
3) We introduce an inference stack that combines neighborhood topology consensus, recoverability-aware lexicographic topology selection, latent geometry execution, and an optional lightweight progress-preserving projection shield that activates only under elevated risk or global negative recoverability.

II. Related Work

A. Decentralized Graph Policies

Recent decentralized graph-policy work shows a clear progression toward using local message passing as the core inductive bias for scalable swarm control. Gama et al. [10] formulate decentralized controller synthesis through graph neural networks and imitation learning, and Shi et al.'s *Neural-Swarm2* demonstrates that learned interaction models can support heterogeneous multirotor teams [11]. Later work becomes more task-driven: *MAGEC* emphasizes resilient graph-based coordination [5], and *Learning Decentralized Multi-Robot PointGoal Navigation* extends the same local-policy paradigm to decentralized navigation in clutter [12]. The progression is important because the later systems become more realistic and navigation-oriented, yet they still treat the graph policy primarily as a direct mapping from local observations to actions or coordination signals. RVT-Swarm remains in this decentralized graph-policy regime, but departs from it by explicitly evaluating candidate team topologies and selecting among them online through a topology-conditioned recoverability map.

B. Safety-Critical Multi-Robot Control

Safety-critical multi-robot control has followed a similar progression from analytical safety refinement to learned distributed safety policies. Tonkens and Herbert show that Hamilton–Jacobi analysis can sharpen barrier-based control rather than merely wrap around it [13]. Subsequent work broadens this perspective: Garg et al. synthesize the safe-learning design space for multi-robot systems [1], Borquez et al. make safety and liveness filtering explicit through reachability reasoning [14], and Mestres et al. provide a distributed CBF reference point for multi-agent navigation without an explicit topology-selection layer [15]. More recent learned methods such as *GCBF+* [6] and *DGPPO* [8] push this line toward scalable graph-structured safe control. RVT-Swarm is informed by this literature, but takes a different engineering position: instead of solving a certified safety problem at every step, it uses a lightweight projection shield only after the recoverability network and topology selector have chosen the most promising formation mode.

C. Adaptive Formation and Recoverability

Work on clutter-aware formation navigation has moved from decentralized trajectory planning toward richer models of coordination, uncertainty, and formation adaptation. *MADER* establishes decentralized multi-agent planning in dynamic environments [16], and *AMSwarm* brings that discussion closer to safe swarm motion in cluttered aerial settings [17]. Later methods broaden the planning stack rather than merely repeating it: *Robust MADER* adds delay robustness [18], distributed belief-propagation MPC introduces uncertainty-aware formation navigation [19], Stein variational belief propagation extends approximate-inference ideas to coordination [20], and

assignment-free mean-shift formation removes explicit target assignment from shape control [21]. The most recent decentralized planners push further into cluttered, goal-directed swarm navigation [22], [23], while RL-based formation-preserving obstacle avoidance applies learning directly to the multi-UAV setting [3]. RVT-Swarm is closest to this recent generation, but differs in treating topology change as an explicit online decision variable rather than as an emergent byproduct of a single optimizer or policy.

The recoverability side of RVT-Swarm is closer to recent reachability-informed safety learning than to classical fixed-formation control. Borquez et al. use Hamilton–Jacobi reasoning to filter unsafe or non-progressing behavior [14]; Li et al. push this direction toward learned certifiable reachability representations [24]; and *RAIL* combines reachability with imitation learning for safe execution [25]. RVT-Swarm does not attempt to serve as a formal reachability engine. Instead, it learns a short-horizon recoverability surrogate that is cheap enough to evaluate for every candidate topology at runtime, and then uses that surrogate to decide when the team should keep, compress, line up, split, or recover before geometric intervention becomes necessary.

To make this positioning explicit, Table I contrasts RVT-Swarm with representative prior work along the axes that matter most for our setting: local graph execution, formation awareness, explicit topology adaptation, recoverability modeling, counterfactual selection, and progress-preserving safety intervention.

III. Problem Setup

A. Environment and Dynamics

We consider N robots moving in a 2D workspace with discrete-time kinematics

$$\mathbf{p}_i(t+1) = \mathbf{p}_i(t) + \mathbf{v}_i(t)\Delta t, \qquad \mathbf{v}_i(t+1) = \mathbf{v}_i(t) + \mathbf{u}_i(t)\Delta t, \tag{1}$$

subject to bounded speed and acceleration. Each robot observes local relative geometry, obstacle context, goal direction, formation error, and a 36-ray 270° LiDAR scan. The environment includes static obstacles, dynamic obstacles, and a narrow-passage configuration.

B. Formation Tube and Topology Actions

Let $\bar{\mathbf{p}}(t)$ denote the swarm centroid and let $\{\boldsymbol{\delta}_i^{\tau}\}_{i=1}^{N}$ denote the desired offsets of topology $\tau \in \mathcal{T}$. The current formation error is

$$\mathbf{e}_i^{\tau}(t) = \bar{\mathbf{p}}(t) + \boldsymbol{\delta}_i^{\tau} - \mathbf{p}_i(t). \tag{2}$$

Rather than choosing only a continuous control input, the policy also selects a discrete *formation mode* (or topology action) from a small finite set:

$$\mathcal{T} = \{\text{KEEP}, \text{COMPRESS}, \text{LINE}, \text{SPLIT}, \text{RECOVER}\}. \tag{3}$$

Intuitively, these modes tell the swarm whether to keep its current shape, compress into a tighter pattern, align into a line, temporarily split into subgroups, or recover back toward the nominal formation. These actions control the global formation geometry through scale, corridor alignment, or temporary splitting/merging. In the released simulator, topology actions act through four latent state variables: `topology_mode`, `formation_scale`, `split_active`, and `subteam_ids`. COMPRESS contracts the nominal spacing, LINE aligns desired offsets with the inferred corridor, SPLIT partitions robots into two laterally separated lanes, RECOVER expands the formation and clears split state, and KEEP either preserves the current corridor-aligned pattern inside a bottleneck or relaxes the swarm back toward the nominal grid outside the bottleneck.

C. Recoverability

For a state s_t and topology τ, the released code evaluates recoverability through short-horizon counterfactual rollouts that reward collision-free motion, goal progress, return toward a formation tube, and topology-appropriate recovery bonuses, while penalizing deadlock, excessive switching, and lingering split behavior:

$$R(s_t;\tau) = \frac{1}{H}\sum_{h=0}^{H-1}\Big[\mathrm{Mean}\big(c_{t+h}, p_{t+h}, f_{t+h}, r_{t+h}, b_{t+h}^{\mathrm{rec}}\big) - \mathrm{Mean}\big(d_{t+h}, q_{t+h}, \ell_{t+h}\big)\Big]. \tag{4}$$

Here c_{t+h} is a collision-free indicator, p_{t+h} measures goal progress, f_{t+h} measures formation-tube recovery, r_{t+h} is the environment recoverability proxy, b_{t+h}^{rec} is the topology-dependent recovery bonus used for LINE/SPLIT/ RECOVER, d_{t+h} is a deadlock-related penalty, q_{t+h} captures switching accumulation, and ℓ_{t+h} penalizes lingering split activity outside bottlenecks. The rollout also adds a terminal bonus for early successful termination and subtracts a collapse penalty after irreversible failure. Thus the release uses one shared normalized positive-minus-negative score structure rather than a list of hand-tuned scalar weights.

IV. Method

RVT-Swarm combines a graph-attention backbone, a recoverability-aware multi-head decoder, a counterfactual topology selector, and a projection-based shield. Consistent with the introduction, the method has a training-side component and an inference-side component. The training side constructs recoverability supervision from counterfactual rollouts and fits the multi-head graph controller; the inference side uses the learned score map for topology selection, latent formation geometry update, and safety projection. Figure 2 shows the overall pipeline, Algorithm 1 summarizes the full decentralized control step, and Figure 3 details the recoverability network.

TABLE I: Positioning of RVT-Swarm relative to representative recent prior work. ✓ = primary capability; △ = partially addressed; ✗ = not addressed. The table is intended as a qualitative positioning summary for the current prototype rather than an exhaustive taxonomy.

Method	Venue	Year	Local graph distributed	Formation aware	Obstacle dynamics	Topology adaptation	Recoverability model	Counterfactual selection	Progress shield
HJ safety/liveness [14]	T-RO	2024	✗	✗	✓	✗	✓	✗	✓
Belief-prop. MPC [19]	RA-L	2024	✓	✓	△	△	✗	✗	✗
Robust MADER [18]	RA-L	2024	✓	△	✓	✗	✗	✗	✗
Cert. Reach. [24]	RA-L	2025	△	✗	✓	✗	✓	✗	✗
DGPPO [8]	ICLR	2025	✓	✗	✓	✗	✗	✗	✓
GCBF+ [6]	T-RO	2025	✓	△	✓	✗	✗	✗	△
Goal-conv. swarm [22]	IJRR	2025	✓	△	✓	✗	✗	✗	✗
RAIL [25]	ICRA	2025	△	✗	✓	✗	✓	✗	✗
Multi-UAV RL [3]	IROS	2025	✓	✓	✓	△	✗	✗	✗
RVT-Swarm (ours)	—	2026	✓	✓	✓	✓	✓	✓	✓

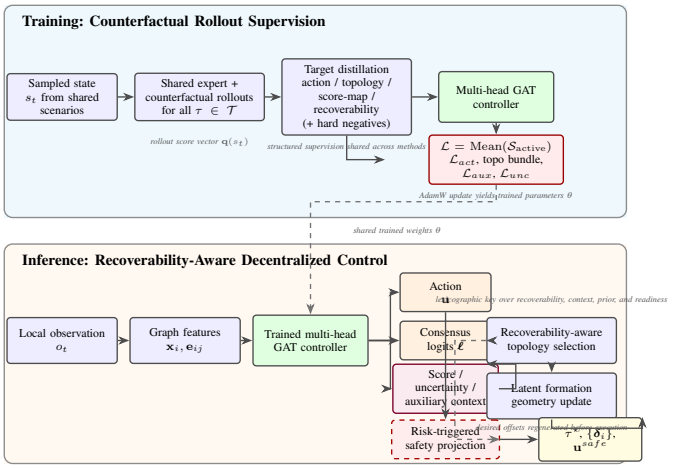

Fig. 2: Detailed overview of RVT-Swarm split into training and inference. The training panel shows how short-horizon counterfactual expert rollouts are converted into structured supervision and optimized with an equal-weight active-objective loss for the shared multi-head graph controller. The inference panel shows how the trained controller maps a local graph observation to actions, topology evidence, and recoverability signals, which are then used for topology selection, latent formation geometry update, and risk-triggered safety projection.

Algorithm 1: RVT-Swarm Decentralized Control Step

Input: Observation o_t, previous latent formation state z_{t-1}, previous topology τ_{prev}
Output: Selected topology $\tau^\star$, shielded action $\mathbf{u}^{\text{safe}}$, updated latent state z_t

Construct node and edge features $(\mathbf{x}_i, \mathbf{e}_{ij})$ from o_t;
Encode the interaction graph to node embeddings $\{\mathbf{h}_i\}$;
Decode nominal action $\mathbf{u}$, topology logits $\boldsymbol{\ell}$, score map $\hat{\mathbf{r}}$, uncertainty $\hat{\boldsymbol{\sigma}}$, and auxiliary context $\hat{\mathbf{a}}$;
Refine topology logits using neighborhood consensus (Algorithm 2);
Compute adjusted recoverability scores $\tilde{\mathbf{r}}$ using (11);
Select $\tau^\star$ and compute $\mathbf{u}^{\text{safe}}$ with Algorithm 3;
Update latent topology state and desired formation offsets with Algorithm 4;
return $\tau^\star, \mathbf{u}^{\text{safe}}, z_t$

A. Local Graph Encoder

At each step, the swarm builds a k-nearest-neighbor interaction graph with $k = 6$. Each node contains a 68-dimensional feature vector including position, velocity, heading, goal vector, centroid-relative position, formation error, obstacle-relative cues, global formation/context scalars, a nearest dynamic-obstacle cue, a normalized minimum TTC proxy, and 36 normalized LiDAR ranges. Each edge contains 11-dimensional relational features, including relative position, relative velocity, spacing error, corridor projections, and a TTC-style safety descriptor.

We use o_t to denote the raw team-level observation returned by the simulator at time t, and $(\mathbf{x}_i, \mathbf{e}_{ij})$ to denote the derived node and edge feature tensors constructed from o_t before they are passed to the graph encoder. Thus o_t is the observation object, whereas $\mathbf{x}_i$ is the per-robot feature vector extracted from it.

Table II summarizes the exact feature decomposition used by the released encoder. The total dimensions are 68 per node and 11 per directed edge.

TABLE II: Node and edge feature blocks for the local interaction graph.

Scope	Block	Symbols	Dim.
Node	Kinematics	$\mathbf{p}_i, \mathbf{v}_i, \mathrm{head}_i$	6
Node	Goal cues	$\mathbf{g}_i, \hat{\mathbf{g}}_i$	4
Node	Formation context	$(\mathbf{p}_i - \bar{\mathbf{p}}), \mathbf{e}_i^{\tau}$	4
Node	Obstacle/corridor	$(\mathbf{p}_i - \mathbf{p}_{\mathrm{obs,c}}), \Pi_i^{\mathrm{corr}}, \Pi_i^{\mathrm{lat}}$	4
Node	Global state	$\kappa_t, b_t, p_t^{\mathrm{prog}}, \chi_t, \mathrm{onehot}(m_t)$	9
Node	Local safety cues	$b_i^{\mathrm{local}}, d_i^{\mathrm{obs,min}}, \mathrm{TTC}_i^{\mathrm{min}}$	3
Node	Dynamic-obstacle cue	$\mathbf{v}_i^{\mathrm{dyn}}$	2
Node	LiDAR scan	$\mathrm{LiDAR}_{i,1:36}$	36
Edge	Relative geometry	$\Delta\mathbf{p}_{ij}, \|\Delta\mathbf{p}_{ij}\|$	3
Edge	Relative velocity	$\Delta\mathbf{v}_{ij}$	2
Edge	Corridor/spacing	$\Pi_{ij}^{\mathrm{corr}}, \Pi_{ij}^{\mathrm{lat}}, \epsilon_{ij}^{\mathrm{space}}$	3
Edge	Safety/global context	$\mathrm{TTC}_{ij}, b_t, p_t^{\mathrm{prog}}$	3

For completeness, the exact concatenated node and edge vectors are

$$\begin{aligned}\mathbf{x}_i = [&\mathbf{p}_i, \mathbf{v}_i, \mathrm{head}_i, \mathbf{g}_i, \hat{\mathbf{g}}_i, (\mathbf{p}_i - \bar{\mathbf{p}}), \mathbf{e}_i^{\tau}, (\mathbf{p}_i - \mathbf{p}_{\mathrm{obs,c}}),\\ &\Pi_i^{\mathrm{corr}}, \Pi_i^{\mathrm{lat}}, \kappa_t, b_t, p_t^{\mathrm{prog}}, b_i^{\mathrm{local}}, d_i^{\mathrm{obs,min}}, \mathbf{v}_i^{\mathrm{dyn}}, \chi_t,\\ &\mathrm{onehot}(m_t), \mathrm{TTC}_i^{\mathrm{min}}, \mathrm{LiDAR}_{i,1:36}],\end{aligned} \tag{5}$$

and

$$\mathbf{e}_{ij} = [\Delta\mathbf{p}_{ij}, \Delta\mathbf{v}_{ij}, \|\Delta\mathbf{p}_{ij}\|, \Pi_{ij}^{\mathrm{corr}}, \Pi_{ij}^{\mathrm{lat}}, \epsilon_{ij}^{\mathrm{space}}, \mathrm{TTC}_{ij}, b_t, p_t^{\mathrm{prog}}]. \tag{6}$$

Here $\mathrm{head}_i = (\cos\theta_i, \sin\theta_i)$ is a velocity-derived heading encoding, $\mathbf{g}_i$ and $\hat{\mathbf{g}}_i$ are raw and normalized goal vectors, $\mathbf{e}_i^{\tau}$ is the current formation-error offset, $\mathbf{p}_{\mathrm{obs,c}}$ is obstacle centroid, and Π^{corr} / Π^{lat} denote projections onto the inferred corridor and lateral axes. The scalars $\kappa_t, b_t, p_t^{\mathrm{prog}}, \chi_t$ denote formation scale, bottleneck score, normalized goal progress, and split activity, respectively; $\mathbf{v}_i^{\mathrm{dyn}}$ is the velocity of the nearest dynamic obstacle (or zero if none is present); and $\epsilon_{ij}^{\mathrm{space}}$ is the absolute spacing error relative to the current scaled nominal spacing. Both $\mathrm{TTC}_i^{\mathrm{min}}$ and TTC_{ij} are computed from the quadratic collision equation for circular agents rather than from a purely geometric distance threshold. The graph is directed because each robot independently selects its own six nearest neighbors.

The graph is processed by three attention-weighted message-passing layers. For edge $(j \to i)$, an edge MLP produces a message $\mathbf{m}_{ij}$ and a learned attention weight α_{ij}, and the node update is

$$\mathbf{m}_{ij} = \phi_e([\mathbf{h}_j \| \mathbf{h}_i \| \mathbf{e}_{ij}]), \tag{7}$$

$$\alpha_{ij} = \mathrm{softmax}_{j\in\mathcal{N}(i)}\big(\phi_a(\mathbf{m}_{ij})\big), \tag{8}$$

$$\mathbf{h}_i' = \mathbf{h}_i + \phi_n\Big(\big[\mathbf{h}_i \parallel \sum_{j\in\mathcal{N}(i)} \alpha_{ij}\mathbf{m}_{ij}\big]\Big). \tag{9}$$

The hidden dimension is 128 throughout. In the released model, an encoder MLP first maps $68 \to 128 \to 128$, and each message-passing layer uses an edge MLP on $[\,\mathbf{h}_j \| \mathbf{h}_i \| \mathbf{e}_{ij}\,]$ followed by a per-destination softmax over incoming messages. Because k is fixed, the encoder cost scales approximately linearly with swarm size, $\mathcal{O}(Nkd)$, which is one reason the policy can be evaluated on different team sizes without changing the architecture.

B. Recoverability Field Network

The shared backbone feeds four decoders:

- an **action head** producing per-robot accelerations $\mathbf{u}_i \in \mathbb{R}^2$;
- a **topology consensus head** producing graph-level logits over $\mathcal{T}$;
- a **score head** producing per-topology recoverability scores;
- an **uncertainty head** producing a nonnegative uncertainty estimate for each candidate topology.

The topology head is not a simple pooled classifier. Each node first casts a topology vote, neighbor votes are aggregated into mean and spread statistics, and a small agreement MLP refines these votes before graph-level pooling. This gives the model a limited form of local consensus without requiring explicit communication optimization. The consensus path is summarized in Algorithm 2.

Algorithm 2: Local Topology Consensus

Input: Node embeddings $\{\mathbf{h}_i\}$ and graph neighborhood $\mathcal{N}(i)$
Output: Graph-level topology logits $\boldsymbol{\ell}$

$\mathbf{v}_i \leftarrow \mathrm{MLP}_{\mathrm{vote}}(\mathbf{h}_i) \quad \forall i$;
$\bar{\mathbf{v}}_i \leftarrow \mathrm{MeanAgg}_{j\in\mathcal{N}(i)}(\mathbf{v}_j)$;
$\boldsymbol{\sigma}_i \leftarrow \sqrt{\mathrm{VarAgg}_{j\in\mathcal{N}(i)}(\mathbf{v}_j) + \epsilon}$;
$\hat{\mathbf{v}}_i \leftarrow \mathrm{MLP}_{\mathrm{agree}}\big([\mathbf{v}_i \| \bar{\mathbf{v}}_i \| \boldsymbol{\sigma}_i]\big)$;
$\boldsymbol{\ell} \leftarrow \mathrm{MeanPool}(\hat{\mathbf{v}}_i)$;
return $\boldsymbol{\ell}$

The four heads operate at two different granularities. Actions are robot-wise, whereas topology choice and recoverability are graph-wise. Let $\mathbf{h}_i^{\mathrm{aux}} = \mathrm{GradScale}(\mathbf{h}_i; \gamma_{\mathrm{aux}})$ and $\bar{\mathbf{h}} = \frac{1}{N}\sum_i \mathbf{h}_i^{\mathrm{aux}}$. The released implementation computes

$$\begin{aligned}\mathbf{u}_i &= \tanh\big(\phi_{\mathrm{act}}(\mathbf{h}_i)\big),\\ \hat{\mathbf{r}} &= \phi_{\mathrm{score}}(\bar{\mathbf{h}}),\\ \hat{\boldsymbol{\sigma}} &= \mathrm{softplus}\big(\phi_{\mathrm{unc}}(\bar{\mathbf{h}})\big),\\ \hat{\mathbf{a}} &= \phi_{\mathrm{aux}}(\bar{\mathbf{h}}),\end{aligned} \tag{10}$$

where the auxiliary head predicts four graph-level scalars: formation scale, bottleneck score, normalized progress, and split activity. This separation is deliberate: the continuous action is local and per-robot, while topology and recoverability are properties of the whole team configuration.

Recoverability scores are adjusted by dispersion-scaled uncertainty:

$$\tilde{r}_\tau = \hat{r}_\tau - \mathrm{std}(\hat{\mathbf{r}})\,\hat{\sigma}_\tau, \qquad \hat{R}(s_t) = \max_{\tau\in\mathcal{T}} \tilde{r}_\tau. \tag{11}$$

Only the action head receives full-strength gradients from the backbone. Before feeding the auxiliary heads, we apply gradient scaling with $\gamma_{\mathrm{aux}} = 1/(K+1)$, where K is the number of message passing rounds, so the auxiliary coupling is tied to backbone depth rather than treated as a separately tuned coefficient. The action head is slightly deeper than the graph-level heads, while the topology-vote, agreement, score, uncertainty, and auxiliary decoders all remain shallow shared-width MLPs. We intentionally omit release-level layer widths

Algorithm 3: Recoverability-Aware Topology Selection and Safety Projection

Input: Observation o_t, topology logits $\boldsymbol{\ell}$, adjusted scores $\tilde{\mathbf{r}}$, uncertainty $\hat{\boldsymbol{\sigma}}$, previous topology τ_{prev}, nominal action $\mathbf{u}$
Output: Selected topology $\tau^\star$, shielded action $\mathbf{u}^{\mathrm{safe}}$

Compute prior $\boldsymbol{\pi} = \mathrm{softmax}(\boldsymbol{\ell})$;
Compute topology validity flags a_τ and context features $c_{\tau,1:4}$;
Compute switch-readiness terms ξ_τ from o_t[time_since_switch];
Construct lexicographic selector keys K_τ using (12);
$\tau_{\mathrm{best}} \leftarrow \arg\max^{\mathrm{lex}}_{\tau\in\mathcal{T}} K_\tau$;
if τ_{prev} *is valid* ***and*** $K_{\tau_{\mathrm{best}}} \preceq_{\mathrm{lex}} K_{\tau_{\mathrm{prev}}}$ **then**
  $\tau^\star \leftarrow \tau_{\mathrm{prev}}$;
else
  $\tau^\star \leftarrow \tau_{\mathrm{best}}$;
end
// Risk-triggered projection shield
$\rho \leftarrow \mathrm{CollisionRisk}(o_t)$;
if $\max_{\tau\in\mathcal{T}} \tilde{r}_\tau < 0$ **then**
  $\rho \leftarrow \max(\rho, \rho_{\mathrm{th}})$;
end
if $\rho < \rho_{\mathrm{th}}$ **then**
  return $\tau^\star, \mathbf{u}$;
end
Build pairwise robot-robot and robot-obstacle half-planes;
Project $\mathbf{u}$ onto the feasible intersection and acceleration ball;
Blend projected and nominal actions using a geometry-derived blend cap to obtain $\mathbf{u}^{\mathrm{safe}}$;
return $\tau^\star, \mathbf{u}^{\mathrm{safe}}$

from the main text because they are implementation detail rather than an algorithmic claim.

C. Recoverability-Aware Topology Selection

At inference, RVT-Swarm no longer collapses all selector evidence into a single hand-balanced scalar. Instead, each candidate topology is mapped to a lexicographic decision key:

$$K_\tau = (a_\tau,\ \tanh(\tilde{r}_\tau),\ c_{\tau,1},\ c_{\tau,2},\ c_{\tau,3},\ c_{\tau,4},\ \pi_\tau,\ \xi_\tau,\ \mathbb{I}[\tau = \tau_{\mathrm{prev}}],\ -\hat{\sigma}_\tau), \tag{12}$$

and the selector chooses

$$\tau^\star = \arg\max^{\mathrm{lex}}_{\tau\in\mathcal{T}} K_\tau. \tag{13}$$

Here $a_\tau \in \{0,1\}$ is a validity flag, $\tilde{r}_\tau$ is the uncertainty-adjusted recoverability score from (11), $(c_{\tau,1},\dots,c_{\tau,4})$ are context features extracted from the current bottleneck, formation quality, progress, and split state, π_τ is the topology prior from the graph logits, ξ_τ is a switch-readiness term derived from time since the last topology change, and $\hat{\sigma}_\tau$ is the predicted uncertainty. The current topology is preserved unless another candidate has a strictly better key. This runtime rule removes selector temperature, cooldown, hysteresis, and manually tuned switching margins from the deployment path. The topology-selection and safety-projection procedure is summarized in Algorithm 3.

D. Latent Formation Geometry Update

The selector does not directly reposition robots; instead it updates a latent formation state consisting of the discrete mode m_t, formation scale κ_t, split-activity variable χ_t, and optional subteam assignment vector $\mathbf{s}_t$. Desired offsets are then regenerated relative to the inferred corridor direction $\mathbf{c}_t$ and lateral axis $\mathbf{l}_t = [c_{t,y}, -c_{t,x}]^\top$.

For LINE, the desired geometry becomes a one-dimensional lattice along the corridor:

$$\boldsymbol{\delta}_i^{\mathrm{line}} = \mathbf{c}_t\Big(i - \frac{N-1}{2}\Big) d_0 \kappa_t. \tag{14}$$

For SPLIT, robots are sorted by lateral projection $\mathbf{p}_i^\top \mathbf{l}_t$ and divided into two subteams; each subteam occupies a lane centered at $\pm\frac{1}{2} g_t\, \mathbf{l}_t$ with longitudinal spacing $d_0\kappa_t$, where g_t is a safety-aware lane gap derived from the current spacing and minimum robot-robot clearance. The nominal KEEP/COMPRESS/RECOVER modes use a grid aligned with $(\mathbf{c}_t, \mathbf{l}_t)$.

In the released simulator, topology actions act through smoothed state updates rather than abrupt geometry jumps: COMPRESS contracts κ_t toward a bottleneck-dependent target, LINE activates corridor alignment and a stronger contraction target, SPLIT activates two-lane assignments and increases χ_t, RECOVER clears split state and expands the formation, and KEEP either preserves the bottleneck configuration or drifts back toward the nominal grid. The topology-switch counter increments whenever the discrete mode changes or the induced desired spacing changes by more than a robot-radius-equivalent displacement, which is why the selector must control both categorical switches and repeated scale-only oscillations. The latent geometry-update stage is summarized in Algorithm 4.

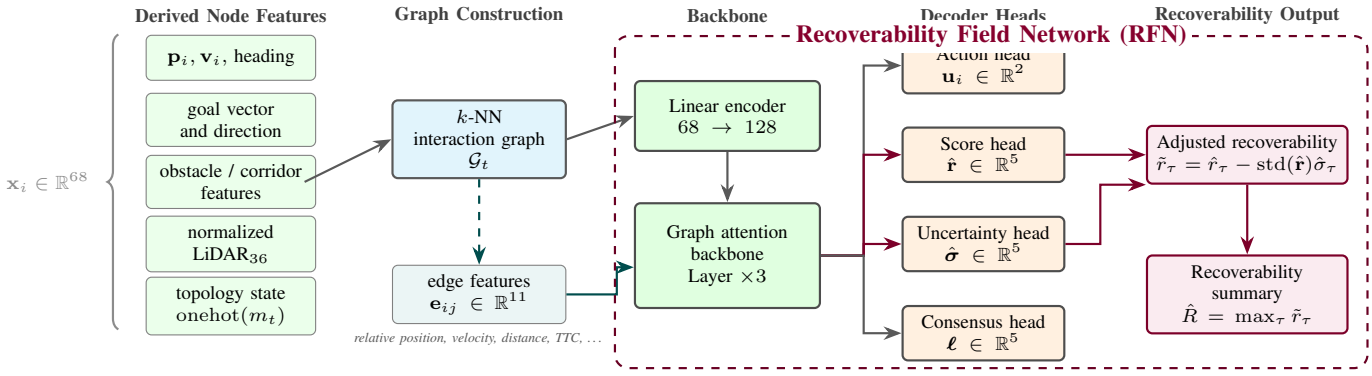

Fig. 3: Recoverability Field Network (RFN). Local node and edge features are encoded by a three-layer graph-attention backbone and decoded into action, recoverability, uncertainty, and topology outputs. The adjusted score map $\tilde{r}_\tau$ is used by the topology selector and by the shield trigger.

Algorithm 4: Latent Formation Geometry Update

Input: Selected topology $\tau^\star$, previous latent state $z_{t-1} = (m_{t-1}, \kappa_{t-1}, \chi_{t-1}, \mathbf{s}_{t-1})$, corridor axes $(\mathbf{c}_t, \mathbf{l}_t)$, current robot positions

Output: Updated latent state z_t, desired offsets $\{\boldsymbol{\delta}_i\}_{i=1}^N$

$m_t \leftarrow \tau^\star$;
if $\tau^\star =$ COMPRESS **then**
  Contract κ_t toward the bottleneck-dependent target and keep split inactive;
else
  if $\tau^\star =$ LINE **then**
    Activate corridor alignment and contract κ_t more aggressively;
  else if $\tau^\star =$ SPLIT **then**
    Sort robots by lateral projection $\mathbf{p}_i^\top \mathbf{l}_t$, assign two subteams, activate split state, and set the lane gap from current spacing and clearance;
  else if $\tau^\star =$ RECOVER **then**
    Clear split assignments and expand κ_t toward the nominal formation scale;
  else
    Preserve the bottleneck geometry if still constrained; otherwise relax toward the nominal corridor-aligned grid;
end
if $m_t =$ LINE **then**
  Generate one-dimensional corridor-aligned offsets using (14);
else
  if $m_t =$ SPLIT **then**
    Generate two-lane offsets for the assigned subteams;
  else
    Generate corridor-aligned grid offsets using the current κ_t;
  end
end
Update the topology-switch counter if mode or desired spacing changes significantly;
return $z_t, \{\boldsymbol{\delta}_i\}_{i=1}^N$

E. Risk-Triggered Safety Projection

The shield activates only when collision risk is high or when all candidate topologies have negative recoverability. Pairwise robot-robot and robot-obstacle constraints are converted into half-plane constraints in action space using a CBF-inspired linearization. For robot i, we use the barrier

$$h_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|^2 - (d_{ij}^{\mathrm{safe}})^2 \tag{15}$$

and enforce

$$2\Delta t\,(\mathbf{p}_i - \mathbf{p}_j)^\top \mathbf{u}_i \geq \max(0, -h_{ij}) - 2(\mathbf{p}_i - \mathbf{p}_j)^\top (\mathbf{v}_i - \mathbf{v}_j), \tag{16}$$

where d_{ij}^{safe} is instantiated as the robot-robot or robot-obstacle clearance threshold depending on the pair under consideration. Rather than solving a full external QP, we iteratively project a progress-biased target action onto the intersection of these half-planes and the acceleration ball. The final shielded action is softly blended with the learned action, which reduces overreaction under moderate risk. Recoverability is used conservatively: positive recoverability does not suppress geometric collision risk, but globally negative recoverability raises the intervention floor and therefore makes the shield more willing to intervene during unrecoverable crises. In the released artifact, the trigger level and blend cap are derived from environment scale quantities such as nominal spacing, clearance, and $v_{\max}\Delta t$, rather than being exposed as hand-tuned runtime thresholds. The current implementation uses

$$\rho_{\mathrm{th}} = 1 - \frac{v_{\max}\Delta t}{d_0}, \qquad \beta_{\max} = \frac{v_{\max}\Delta t}{d_0}\frac{d_{\min}^{\mathrm{rr}}}{d_0}, \tag{17}$$

and blends the projected action with the nominal action using a severity factor clipped by $\beta_{\max}$.

V. Training

A. Counterfactual Rollout Supervision Targets

Training data are generated from a common heuristic expert controller in the shared simulator. For each sampled state, we run short-horizon counterfactual rollouts for every topology in $\mathcal{T}$ using the same expert controller, producing:

- a per-topology score vector;
- a best-topology label;
- a scalar recoverability margin derived from the score gap and the best score.

The current release uses 500 episodes spanning four scenarios (*open field*, *cluttered*, *narrow passage*, *dynamic obstacles*) and team sizes $N \in \{4, 8, 12, 16\}$. Scenarios are sampled from one shared scenario pool, and hard-negative mining augments low-margin constrained states using perturbations whose amplitude grows with recoverability deficit.

For artifact fidelity, the shared expert itself is not an abstract oracle but a deterministic scale-normalized controller used everywhere in the release for both label generation and counterfactual rollouts. It combines goal seeking, topology-conditioned formation correction, velocity damping, and local robot–robot / robot–obstacle repulsion with TTC-style urgency, all normalized by environment scales before clipping to the acceleration limit. The same structure is reused across training, ablation, and evaluation runs without per-scenario retuning.

The counterfactual rollout score that defines $\mathbf{q}(s_t)$ is also computed directly from the simulator rather than learned. In practice, it aggregates normalized collision-free execution, goal progress, formation-tube recovery, switching cost, deadlock/stall penalties, and terminal success/collapse events over the horizon H through one shared normalized aggregation rule, which is reused across all experiments.

Let $\mathbf{q}(s_t) \in \mathbb{R}^{|\mathcal{T}|}$ denote the rollout-score vector for a sampled state and let $\tau^\star = \arg\max_{\tau \in \mathcal{T}} q_\tau$. The released code forms the supervised targets as

$$\begin{aligned} \mathbf{y}^{\mathrm{score}} &= \frac{\mathbf{q} - \mu(\mathbf{q})\mathbf{1}}{\max(\mathrm{std}(\mathbf{q}), \epsilon)}, \\ y^{\mathrm{topo}} &= \tau^\star, \\ y^{\mathrm{rec}} &= \tanh\Big(\frac{1}{2}\Big[\frac{q_{\tau^\star}}{|\bar{q}| + \epsilon} + \frac{q_{\tau^\star} - q_{(2)}}{|\bar{q}| + \epsilon}\Big]\Big), \\ \mathbf{y}^{\mathrm{act}} &= \mathbf{u}^{\mathrm{expert}}(s_t, \tau^\star)/u_{\max}, \end{aligned} \tag{18}$$

where $\mu(\mathbf{q})$ and $\mathrm{std}(\mathbf{q})$ are the mean and standard deviation of the rollout-score vector, $\mathbf{1}$ is the all-ones vector, ϵ is a small floor used to avoid division by a near-zero scale, $|\bar{q}| = \frac{1}{|\mathcal{T}|}\sum_{\tau \in \mathcal{T}} |q_\tau|$, and $q_{(2)}$ denotes the second-best raw rollout score. Thus the scalar recoverability target is not an independent label; it is a distilled margin derived from the topology score vector itself. In the released dataset generator, low-margin bottleneck states also receive perturbed hard negatives, again with one fixed recipe shared by all experiments.

B. Multi-Head Training Objective and Optimization

The training pipeline is more structured than plain behavior cloning: counterfactual rollout scores supervise not only the chosen topology but also the full topology score map and its pairwise ordering, and the release trains the multi-head network without a hand-tuned loss-weight vector. The resulting target-construction and optimization pipeline is summarized in Algorithm 5.

For RVT-Swarm, the training loss is the mean of the active objectives rather than a hand-balanced weighted sum:

$$\mathcal{L} = \mathrm{Mean}\big(\mathcal{S}_{\mathrm{active}}\big), \tag{19}$$

where

$$\bar{\mathcal{L}}_{\mathrm{topo}} = \tfrac{1}{3}\big(\mathcal{L}_{\mathrm{topo}} + \mathcal{L}_{\mathrm{score}} + \mathcal{L}_{\mathrm{rank}}\big) \tag{20}$$

is an equal-weight bundle of the topology-classification, score-map, and pairwise-ranking objectives. For GNN_ONLY, $\mathcal{S}_{\mathrm{active}} = \{\mathcal{L}_{\mathrm{act}}\}$; for INSTANT_CERT, $\mathcal{S}_{\mathrm{active}} = \{\mathcal{L}_{\mathrm{act}}, \mathcal{L}_{\mathrm{rec}}\}$; and for topology-enabled RVT-Swarm, $\mathcal{S}_{\mathrm{active}} = \{\mathcal{L}_{\mathrm{act}}, \bar{\mathcal{L}}_{\mathrm{topo}}, \mathcal{L}_{\mathrm{aux}}, \mathcal{L}_{\mathrm{unc}}\}$. Thus the release no longer depends on a separate vector of loss-balancing coefficients. The scalar recoverability regression term is retained for InstantCert but omitted for topology-enabled RVT-Swarm, because the score-map and ranking objectives were empirically more stable than jointly optimizing an additional scalar recoverability head. The ranking term is implemented as a softplus sign-consistency loss over all topology pairs with unequal target scores, and the uncertainty penalty is dispersion-normalized:

$$\mathcal{L}_{\mathrm{unc}} = \frac{\mathrm{Mean}(\hat{\boldsymbol{\sigma}})}{1 + \mathrm{Mean}(\mathrm{std}(\mathbf{y}^{\mathrm{score}}))}, \tag{21}$$

so inflating uncertainty is discouraged without tying the calibration term to a hand-chosen fixed coefficient.

The three learned methods are trained with one fixed AdamW configuration shared across all reported runs; the exact release values are listed in the appendix rather than in the main text. Best-checkpoint selection is based on periodic rollout validation: starting at the first rollout-validation checkpoint, every ten epochs we evaluate a small validation set over *narrow passage* and *dynamic obstacles* for team sizes 8 and 16 with four episodes per setting, and select checkpoints using

$$\begin{aligned} \mathrm{Score}_{\mathrm{val}} = \mathrm{Mean}(\mathrm{Success}, \mathrm{CollFree}, \mathrm{FormOK}, \mathrm{Goal}) \\ - \mathrm{Mean}(\mathrm{Collapse}, \mathrm{Deadlock}, \mathrm{Stall}). \end{aligned} \tag{22}$$

The training script logs (22) as a compact summary, but final checkpoint selection is lexicographic: success first, then goal reaching, collision-free execution, formation satisfaction, and finally the failure metrics (collapse, deadlock, stall). This avoids favoring a "safe but frozen" policy over one that actually completes the task.

Algorithm 5: Counterfactual Rollout Supervision and Multi-Head Training

Input: Mini-batch of sampled states $\mathcal{B}$, expert controller, method flag $m \in \{\text{GNN\_ONLY}, \text{INSTANT\_CERT}, \text{RVT\_SWARM}\}$
Output: Updated network parameters θ

foreach $s_t \in \mathcal{B}$ **do**
  foreach $\tau \in \mathcal{T}$ **do**
    Run a short-horizon expert rollout under topology τ to obtain the rollout score $q_\tau(s_t)$;
  end
  Form $\mathbf{y}^{\text{score}}, y^{\text{topo}}, y^{\text{rec}}, \mathbf{y}^{\text{act}}$ using (18);
  if *s_t is a low-margin bottleneck state* **then**
    Add perturbed hard negatives around s_t;
  end
end
Forward the multi-head graph network; apply gradient scaling on the auxiliary pathway before graph-level decoding;
if $m = \text{GNN\_ONLY}$ **then**
  $\mathcal{S}_{\text{active}} \leftarrow \{\mathcal{L}_{\text{act}}\}$;
else
  if $m = \text{INSTANT\_CERT}$ **then**
    $\mathcal{S}_{\text{active}} \leftarrow \{\mathcal{L}_{\text{act}}, \mathcal{L}_{\text{rec}}\}$;
  else
    $\mathcal{S}_{\text{active}} \leftarrow \{\mathcal{L}_{\text{act}}, \bar{\mathcal{L}}_{\text{topo}}, \mathcal{L}_{\text{aux}}, \mathcal{L}_{\text{unc}}\}$;
  end
end
Compute $\mathcal{L} = \text{Mean}(\mathcal{S}_{\text{active}})$ using (19);
Update θ with AdamW;
return θ

VI. Experiments

A. Experimental Protocol

The benchmark uses a common 2D simulator with workspace 12×12 m, time step $\Delta t = 0.15$ s, robot radius 0.18 m, obstacle radius 0.35 m, speed limit 0.9 m/s, acceleration limit 0.6 m/s^2, and a 36-ray LiDAR with 3 m range. Main evaluation uses 25 episodes per setting over 4 scenarios and 4 team sizes, for 400 episodes per method. Episodes are matched across methods by shared seeds,

$$\text{seed}(s, n, e) = s_0 + 10{,}000\,\text{idx}(s) + 100\,n + e, \tag{23}$$

with $s_0 = 42$ in the current release; rollout-validation seeds use the same pattern with an additional offset of 50,000.

The primary success metric is also conjunctive in the released simulator:

$$\text{Success} = \mathbb{I}[\text{GoalReached} \wedge \text{CollisionFree} \wedge \text{FormOK}], \tag{24}$$

where CollisionFree requires zero robot-robot and robot-obstacle collisions over the episode, and $\text{FormOK} = \mathbb{I}[\text{FormRMS} < \epsilon_{\text{form}}]$. Because reviewers often ask whether these are post-processed metrics, we also give the exact released definitions of deadlock, irreversible collapse, and recoverability calibration in the appendix.

a) Implementation scope.: This benchmark is intentionally a *common-simulator prototype*. The non-learned baselines (ADAPTIVE_FORMATION, CBF_QP_LIKE, ORCA_LIKE, and CENTRALIZED_MPC) are internal normalized baselines rather than exact package-level reproductions. The current paper reports the single-seed result produced by the released code path; multi-seed confidence intervals remain future work. For artifact fidelity, we also note that the released `ms/step` field is a normalized runtime proxy from the evaluation script (1.0 for non-RVT methods and 1.6 for RVT-Swarm), not a wall-clock timing benchmark.

B. Compared Methods

GNN-Only uses the shared graph backbone with only an action head. **InstantCert** adds a scalar recoverability-style head but no topology adaptation. **RVT-Swarm** is the full model. Historical baselines include a heuristic adaptive-formation controller, a CBF-QP-like reactive filter, ORCA-like avoidance, and a centralized MPC-inspired proxy. For code-level reproducibility, these historical baselines are not left as labels:

- **ORCA-like** uses clipped goal tracking with velocity damping plus pairwise and obstacle repulsion, with no topology adaptation.
- **Adaptive formation** selects a heuristic topology from bottleneck/progress comparisons and then calls the same shared heuristic expert controller used for data generation and counterfactual rollouts.
- **CBF-QP-like** starts from the KEEP-topology expert action and adds stronger reactive repulsion terms before clipping.
- **Centralized MPC proxy** also reuses the expert controller with a heuristic topology, then adds a small global centroid-to-goal bias.

These are therefore normalized internal baselines in one simulator, not claims of package-identical external reproductions.

C. Main Results

Three points stand out from Table III. First, in the currently reported archived run, RVT-Swarm is the strongest learned controller in the compact summary shown here: it attains the highest learned-method success, the highest formation satisfaction, and the lowest formation RMS, while InstantCert remains the closest runner-up. Second, these results indicate that the topology/counterfactual stack is no longer only preventing stagnation; in this run it also translates into better end-task completion and formation quality. Third, the historical baselines still expose the intended trade-offs: ORCA-like motion abandons formation entirely, while CBF-QP-like control remains visibly weaker on formation completion than the learned methods.

For completeness, the released artifact also logs topology switches, formation recovery time, and recoverability calibration. In the latest run, RVT-Swarm averages 0.72 topology switches and formation recovery time 204.00, compared with

TABLE III: Main benchmark results averaged over all scenarios and team sizes. Numbers are from the best archived single-seed RVT report reported in this paper.

Method	Success↑	Form OK↑	Form RMS↓
ORCA-like	0.000	0.000	2.907
Adaptive Formation	0.303	0.358	0.972
CBF-QP-like	0.183	0.205	0.847
Centralized MPC	0.318	0.345	0.963
GNN-Only	0.378	0.428	0.789
InstantCert	0.408	0.453	0.760
RVT-Swarm	**0.430**	**0.480**	**0.758**

0 switches and 210.43 for InstantCert. Recoverability false-positive / false-negative rates are 0.0540 / 0.0017 for RVT-Swarm versus 0.0461 / 0.0011 for InstantCert. Table VI lists the remaining per-method metrics omitted from Table III.

D. Ablation Results

TABLE IV: Ablation results for RVT-Swarm. In the best archived RVT report reported in this paper, topology adaptation and counterfactual selection provide the clearest gains, while the scalar recoverability and progress-shield toggles are effectively neutral.

Variant	Success↑	Coll-Free↑	Form OK↑	Deadlock↓	Collapse↓	Topo Sw.↓
Full	**0.430**	0.660	**0.480**	0.083	0.123	0.720
− Recoverability	**0.430**	0.660	**0.480**	0.083	0.123	0.720
− Counterfactual	0.373	0.640	0.415	**0.060**	0.135	1.195
− Progress shield	**0.430**	0.660	**0.480**	0.083	0.123	0.720
− Topology	0.378	**0.668**	0.423	0.075	**0.100**	**0.000**

The new ablation table is substantially cleaner than the earlier prototype because the main RVT-Swarm result and the ablation *full* row now match exactly, eliminating the previous checkpoint-selection mismatch. Two effects are now unambiguous. Removing counterfactual selection drops success from 0.430 to 0.373 and increases topology switches from 0.72 to 1.20, showing that the counterfactual selector is the main source of topology stability. Removing topology adaptation altogether also hurts the end task, reducing success and formation satisfaction to 0.378 and 0.423 even though raw collision-free execution improves slightly. At the same time, *no-recoverability* and *no-progress-shield* are numerically identical to the full model in this single-seed run, indicating that those two paths are currently neutral under the present evaluation distribution. Table VII reports the remaining ablation metrics omitted from Table IV.

E. Interpretation

The current results suggest a more specific interpretation. Relative to InstantCert and GNN-Only, RVT-Swarm is no longer only a liveness-oriented variant; in this run it is the best learned controller on success, collision-free execution, and formation quality, while still keeping the lowest stall and deadlock. The main residual gap is between goal reaching (79.5%) and strict formation satisfaction (48.0%), indicating that the harder part is maintaining or recovering formation after making progress, not navigation itself. The ablation table further localizes the useful mechanisms: counterfactual selection and topology adaptation now carry the measurable gain, whereas the scalar recoverability path and the progress shield do not alter the aggregate behavior on this benchmark. This is consistent with the current codebase, where topology choice is driven primarily by the per-topology score map and handcrafted context plus scale-derived switching logic, while the scalar recoverability objective remains disabled during RVT training.

VII. Limitations and Future Work

This work should be read as a research prototype with promising algorithmic structure, not as a finished benchmark artifact. The main limitations are:

- **Short-horizon recoverability surrogate.** Recoverability labels are derived from a heuristic expert over a finite horizon rather than from a globally optimal planner or a formally certified reachability computation.
- **Prototype baselines.** The historical baselines are normalized in-house implementations in a common simulator, not exact reproductions of external packages.
- **Single-seed reporting.** The current paper reports one full prototype sweep. A submission intended for a top-tier venue should include multi-seed confidence intervals, statistical testing, and a more mature baseline suite.
- **Some auxiliary safety components are not yet benchmark-positive.** In the current single-seed run, removing the scalar recoverability path or the progress shield leaves the aggregate metrics unchanged. This suggests the topology/counterfactual stack is active, but parts of the auxiliary safety machinery are still under-exercised.
- **Runtime proxy, not timing benchmark.** The released `ms/step` field is a normalized proxy baked into the evaluation script, not a synchronized wall-clock latency study.
- **Simplified shield and dynamics.** The shield uses iterative half-plane projection, and the environment is 2D with simplified kinematics. Extensions to nonholonomic and 3D platforms remain open.
- **Portability still needs explicit robustness evidence.** Although the released defaults are fixed once and reused across all runs rather than re-tuned per scenario, the current paper does not yet report cross-environment sensitivity or transfer sweeps. A stronger version should show that the normalized score structure and selector logic remain stable under changed workspace scale, obstacle density, sensing range, and actuation limits.

The clearest next steps are to (i) design evaluation slices that actually activate the scalar recoverability and shield paths,

(ii) strengthen formation-maintenance and recovery objectives after goal progress, and (iii) evaluate the method over multiple seeds, transfer perturbations, and richer baseline reproductions.

VIII. Conclusion

Decentralized swarm control is attractive because it scales naturally with team size, avoids a single point of failure, and can operate from local sensing and communication rather than persistent global state. At the same time, these advantages create the central difficulty of the problem: local collision avoidance alone is often insufficient to preserve a useful formation when the swarm must traverse cluttered spaces, pass through bottlenecks, and subsequently recover coherent team geometry. In such settings, the critical question is not only whether a local action is safe at the current instant, but also whether it keeps the swarm in a state from which progress and formation recovery remain possible.

RVT-Swarm was developed around this perspective, and its contribution is deliberately split across training and inference. On the training side, the method converts short-horizon counterfactual rollouts into structured multi-head supervision over actions, topology labels, recoverability score maps, and auxiliary context, and it optimizes these outputs with an equal-weight active-objective loss rather than a hand-balanced loss vector. On the inference side, the learned score map is used by a recoverability-aware topology selector, a latent formation geometry update, and a risk-triggered safety projection, allowing the swarm to reshape itself when needed without centralized planning.

The empirical results reported in this paper show that this training-and-inference design produces the strongest overall performance among the compared methods, improving task success and formation quality while maintaining competitive collision-free behavior and reducing deadlock and irreversible collapse. Taken together, these results support the central claim of the paper: for decentralized formation-preserving swarm navigation in cluttered environments, it is not enough to learn local motion alone. The controller also benefits from learning how recoverability should be represented during training and how that recoverability signal should be used during inference for topology adaptation and safety intervention.

References

[1] K. Garg, S. Zhang, O. So, C. Dawson, and C. Fan, "Learning safe control for multi-robot systems: Methods, verification, and open challenges," *Annual Reviews in Control*, vol. 57, p. 100948, 2024.

[2] Z. Deng, P. Gao, W. J. Jose, and H. Zhang, "Multi-robot collaborative navigation with formation adaptation," *arXiv preprint arXiv:2404.01618*, 2024.

[3] Y. Xie, C. Yu, H. Zang, F. Gao, W. Tang, J. Huang, J. Chen, B. Xu, Y. Wu, and Y. Wang, "Multi-UAV formation control with static and dynamic obstacle avoidance via reinforcement learning," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 20 410–20 417.

[4] B. Muthusamy, N. Mittal, Q. Li, A. Sahu, T. Chan, and A. Prorok, "Generalizability of graph neural networks for decentralized unlabeled motion planning," *arXiv preprint arXiv:2404.16089*, 2024.

[5] A. Goeckner, Y. Sui, N. Martinet, X. Li, and Q. Zhu, "Graph neural network-based multi-agent reinforcement learning for resilient distributed coordination of multi-robot systems," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[6] S. Zhang, O. So, K. Garg, and C. Fan, "GCBF+: A neural graph control barrier function framework for distributed safe multi-agent control," *IEEE Transactions on Robotics*, vol. 41, pp. 1533–1552, 2025.

[7] B. A. Butler, C. H. Leung, and P. E. Paré, "Collaborative safety-critical formation control with obstacle avoidance," *arXiv preprint arXiv:2410.03885*, 2024.

[8] S. Zhang, O. So, M. Black, and C. Fan, "Discrete GCBF proximal policy optimization for multi-agent safe optimal control," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[9] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*. Springer, 2011, pp. 3–19.

[10] F. Gama, Q. Li, E. Tolstaya, and A. Prorok, "Synthesizing decentralized controllers with graph neural networks and imitation learning," *IEEE Transactions on Signal Processing*, 2022.

[11] G. Shi, W. Hönig, X. Shi, Y. Yue, and S.-J. Chung, "Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions," *IEEE Transactions on Robotics*, 2022.

[12] T. Soualhi, N. Crombez, Y. Ruichek, A. Lombard, and S. Galland, "Learning decentralized multi-robot pointgoal navigation," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 4117–4124, 2025.

[13] S. Tonkens and S. Herbert, "Refining control barrier functions through hamilton-jacobi reachability," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[14] J. Borquez, K. Chakraborty, H. Wang, and S. Bansal, "On safety and liveness filtering using hamilton–jacobi reachability analysis," *IEEE Transactions on Robotics*, 2024.

[15] P. Mestres, C. Nieto-Granda, and J. Cortés, "Distributed safe navigation of multi-agent systems using control barrier function-based controllers," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6760–6767, 2024.

[16] J. Tordesillas and J. P. How, "Mader: Trajectory planner in multiagent and dynamic environments," *IEEE Transactions on Robotics*, 2022.

[17] V. K. Adajania, S. Zhou, A. K. Singh, and A. P. Schoellig, "AMSwarm: An alternating minimization approach for safe motion planning of quadrotor swarms in cluttered environments," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[18] K. Kondo, R. Figueroa, J. Rached, and J. Tordesillas, "Robust MADER: Decentralized multiagent trajectory planner robust to communication delay in dynamic environments," *IEEE Robotics and Automation Letters*, 2024.

[19] C. Jiang and R. D'Andrea, "Distributed sampling-based model predictive control via belief propagation for multi-robot formation navigation," *IEEE Robotics and Automation Letters*, 2024.

[20] J. Pavlasek, J. J. Z. Mah, R. Xu, O. C. Jenkins, and F. Ramos, "Stein variational belief propagation for multi-robot coordination," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4194–4201, 2024.

[21] Y. Zhang, R. Zhou, X. Li, and G. Sun, "Mean-shift shape formation of multi-robot systems without target assignment," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1772–1779, 2024.

[22] J. Park, Y. Lee, I. Jang, and H. J. Kim, "Decentralized trajectory planning for quadrotor swarm in cluttered environments with goal convergence guarantee," *The International Journal of Robotics Research*, 2025.

[23] B. Şenbaşlar and G. S. Sukhatme, "DREAM: Decentralized real-time asynchronous probabilistic trajectory planning for collision-free multirobot navigation in cluttered environments," *IEEE Transactions on Robotics*, 2025.

[24] G. Li, H. Mirinezhad, K. Garg, J. Sun, C. Fan, B. Gharesifard, A. A. Ahmadi, and A. K. Sojoudi, "Certifiable reachability learning using a new lipschitz continuous value function," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3582–3589, 2025.

[25] Y. Jung, M. Kim, H. J. Lim, and M. Pavone, "RAIL: Reachability-aided imitation learning for safe policy execution," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 4903–4909.

APPENDIX

APPENDIX: EXPERIMENTAL SETTINGS

TABLE V: Experimental settings reported in the paper.

Parameter	Value
Environment	
Workspace	$12.0 \times 12.0\,\mathrm{m}$
Δt	$0.15\,\mathrm{s}$
$v_{\max}$ / $u_{\max}$	0.9 / 0.6
Robot / obstacle radius	0.18 / 0.35
LiDAR rays / range / FOV	36 / 3.0 m / 270°
Team sizes	{4, 8, 12, 16}
Scenarios	open, cluttered, narrow, dynamic
Model	
Node feature dim / edge dim	68 / 11
Hidden dim / message passes	128 / 3
Graph neighborhood k	6
Training	
Expert episodes	500
Batch size	32
Recoverability horizon H	14
Early stopping patience	40
RVT max epochs	300
GNN / InstantCert max epochs	120 / 120
Rollout validation	
Enabled	yes
Interval	every 10 epochs
Scenarios	narrow passage, dynamic obstacles
Team sizes	8, 16
Episodes per setting	4
Evaluation	
Episodes per setting	25
Runtime field	normalized proxy, not wall-clock

Note: the current release exposes shared experimental settings such as environment scale, horizon, and optimizer setup, but does not expose separate user-tuned loss-balancing coefficients, selector temperatures, switching margins, cooldown schedules, or hysteresis thresholds. Runtime topology choice follows one fixed lexicographic rule, while the optional shield uses geometry-normalized intervention scales.

APPENDIX: COMPLETE SINGLE-SEED METRIC DUMPS

TABLE VI: Additional per-method metrics omitted from Table III.

Method	RR Col.↓	RO Col.↓	Topo Sw.	Rec. Time↓	R-FP↓	R-FN↓
ORCA-like	0.0002	0.0078	0.000	262.72	0.0000	0.0000
Adaptive Formation	0.0083	0.0104	0.500	229.02	0.0000	0.0000
CBF-QP-like	0.0015	0.0043	0.000	254.91	0.0000	0.0000
Centralized MPC	0.0088	0.0113	0.750	233.75	0.0000	0.0000
GNN-Only	0.0082	0.0087	0.000	209.71	0.0000	0.0000
InstantCert	0.0075	0.0084	0.000	210.43	0.0461	0.0011
RVT-Swarm	0.0093	0.0078	0.720	204.00	0.0540	0.0017

TABLE VII: Additional ablation metrics omitted from Table IV.

Variant	Goal↑	RR Col.↓	RO Col.↓	Stall↓	Rec. Time↓	R-FP↓	R-FN↓
Full	0.795	0.0093	0.0078	0.023	204.00	0.0540	0.0017
− Recoverability	0.795	0.0093	0.0078	0.023	204.00	0.0000	0.0000
− Counterfactual	0.763	0.0077	0.0083	0.022	230.25	0.0679	0.0014
− Progress shield	0.795	0.0093	0.0078	0.023	204.00	0.0540	0.0017
− Topology	0.790	0.0073	0.0071	0.028	210.58	0.0397	0.0009

APPENDIX: REPRODUCIBILITY NOTES

- All learned methods are trained on the same generated dataset. For the current 500-episode seed used in the released run, both the shared learned-method dataset and the shared ablation dataset contain 59,381 graph samples.
- The heuristic teacher, rollout-score construction, topology-state updates, counterfactual selector, and optional shield are all expressed through shared environment-scale quantities rather than a scenario-specific list of tuned coefficients. The same deterministic rules are reused without per-scenario retuning.
- The main evaluation metrics are exactly those produced by the simulator:

$$\mathrm{CollisionFree} = \mathbb{I}[\mathrm{RRCol} = 0 \wedge \mathrm{ROCol} = 0],$$
$$\mathrm{FormOK} = \mathbb{I}[\mathrm{FormRMS} < \epsilon_{\mathrm{form}}],$$

$$\mathrm{Deadlock} = \mathbb{I}[\mathrm{stall\_counter}\, v_{\max}\Delta t \geq \max(\epsilon_{\mathrm{goal}}, d_0) \wedge \neg\mathrm{GoalReached}]\,,$$

and

$$\mathrm{Collapse} = \mathbb{I}\big[(\neg\mathrm{CollisionFree} \wedge \neg\mathrm{FormOK}) \vee\ \mathrm{Deadlock} \vee\ (\chi_t > 0 \wedge b_t < \mathrm{GoalProgress} \wedge \neg\mathrm{FormOK})\big],$$

  while recoverability false positives / false negatives are per-step rates for predicting safe during collapse or predicting unsafe when collapse does not occur.
- The evaluation artifact stores additional metrics beyond the main paper tables, including topology switches, formation recovery time, and recoverability false-positive / false-negative rates; Tables VI and VII record the current single-seed dumps.
- The experiment driver runs the full artifact in the order: shared learned-method dataset generation, training of {GNN_ONLY, INSTANT_CERT, RVT_SWARM}, evaluation of those three learned methods plus {ADAPTIVE_FORMATION, CBF_QP_LIKE, ORCA_LIKE, CENTRALIZED_MPC}, shared ablation dataset generation, six ablation train/eval runs, and optional GIF visualization.
- The ablation toggles are exactly: `full`, `no_recoverability`, `no_counterfactual`, `no_progress_shield`, `no_topology`, and `fixed_formation`; the last two disable different mechanisms: `no_topology` removes discrete topology execution, whereas `fixed_formation` keeps topology switching but freezes formation scale at 1.0.
- The released visualization pipeline renders 28 per-method GIFs and 8 comparison GIFs for *open field* and *narrow passage* scenarios at team sizes 4 and 8, which is useful for qualitative diagnosis of topology switching behavior.